DATATHON MICAI 2026

Reto 3 | Modelo de otorgamiento de crédito

Empresa participante: ROGA

Construir un modelo de clasificación binaria que prediga, **al momento de la solicitud del crédito**, si el cliente liquidará su crédito dentro del plazo estipulado o fuera de él.

Variable objetivo: `cumple_plazo` (1 = liquidó dentro del plazo, 0 = liquidó fuera del plazo)

El modelo debe usar únicamente información disponible al momento de la solicitud. No se permite el uso de variables que se generan después de otorgar el crédito (como historial de pagos, saldo actual, días sin abonar, etc.).

---

1. Contexto de negocio

La empresa otorga créditos a plazo (99.7% a 20 meses) para la compra de productos. Del total de créditos liquidados en el conjunto de datos, el **34.9% se liquidó fuera del plazo** estipulado. Un modelo predictivo permitiría:

- Priorizar la investigación de solicitantes con alta probabilidad de incumplimiento.
- Ajustar políticas de otorgamiento (enganche, plazo, documentación adicional).
- Reducir la cartera vencida y mejorar la salud financiera de la operación.

---

2. Datos entregados

Recurso	Archivo en este proyecto	Descripción
Dataset principal	`datos/datathon_dataset_entrega.csv`	5,935 créditos liquidados, 24 columnas (20 features de originación + target + columnas de referencia)
Diccionario de datos	`datos/datathon_diccionario_datos.csv`	Columna, tipo, categoría y descripción de cada variable

Recurso	Archivo en este proyecto	Descripción
Rúbrica oficial	`evaluacion/DATATHON_2026_Rubrica_ROGA.docx`	Documento original de descripción del reto y rúbrica

```
None
roga/
├── README.md
├── datos/
│   ├── datathon_dataset_entrega.csv
│   └── datathon_diccionario_datos.csv
└── evaluacion/
    └── DATATHON_2026_Rubrica_ROGA.docx
```

Resumen del dataset

- Registros: 5,935 créditos liquidados, con `fecha_entrega` entre enero y diciembre de 2023.
- Balance de clases: 65.1% cumplió el plazo, 34.9% no cumplió.
- Los datos están anonimizados: identificadores transformados e ingresos normalizados.

2.1 Features numéricas

Columna	Descripción
`Importe_Pedido`	Monto total del pedido
`pago_quincenal`	Cuota quincenal del crédito
`monto_financiado`	Importe del pedido menos el anticipo
`Anticipo_Condicionado`	Monto del enganche/adelanto (0 = sin anticipo)
`ratio_anticipo`	Anticipo / Importe del pedido
`plazo_meses`	Plazo del crédito en meses
`n_productos`	Número de artículos en el pedido
`Ingreso_Cliente`	Ingreso declarado del solicitante (normalizado)
`Meses_Empleo`	Antigüedad laboral en meses
`Capacidad_Quincenal`	Ingreso menos compromisos financieros

Columna	Descripción
`edad_credito`	Edad del cliente al solicitar (años)
`personas_en_Domicilio`	Personas en el domicilio
`sueldo_conyuge`	Sueldo del cónyuge (normalizado, 0 = sin cónyuge)
`meses_antiguedad_conyuge`	Antigüedad laboral del cónyuge en meses
`sd1`	Variable producto de una lógica de negocio
`Tiene_Aval`	1 = tiene aval, 0 = no tiene

2.2 Features categóricas

Columna	Valores posibles
`Tipo_Cliente`	Cliente Anterior, Cliente Nuevo
`estado_civil`	Dato que mapea el estado civil
`Tipo_Domicilio`	Dato que mapea el tipo de domicilio
`Tipo_Empleo`	Dato que mapea el tipo de empleo
`Tipo_Empleo_Conyuge`	Tipo de empleo del cónyuge; vacío si no tiene empleo o no fue capturado

2.3 Columnas adicionales

Columna	Descripción
`cumple_plazo`	**TARGET**. 1 = liquidó en plazo, 0 = fuera de plazo
`exceso_dias`	Días de exceso sobre el plazo (negativo = pagó antes, positivo = se pasó). Referencia informativa, **NO usar como feature**
`fecha_entrega`	Fecha de entrega del crédito (usar para split temporal)

2.4 Notas importantes

- El 99.7% de los créditos son a 20 meses de plazo.
- Algunos campos de texto traen espacios al final (por ejemplo `Tipo_Cliente`); la limpieza es parte del reto.
- El split temporal sugerido es **80/20 por `fecha_entrega`**.

---

3. Rúbrica de evaluación

Fórmula de calificación: `(EDA × 0.30) + (Modelo × 0.40) + (Arquitectura × 0.30) + Bonus`

3.1 Comprensión de datos y EDA (30%)

Nivel	Puntos	Descripción
Sobresaliente	90–100	EDA exhaustivo: detecta distribución multimodal de días de pago, identifica variables clave (ej. `Capacidad_Quincenal`), visualizaciones claras y bien interpretadas, manejo correcto de datos faltantes, análisis de correlaciones y asociaciones categóricas
Notable	75–89	EDA completo con visualizaciones relevantes, identifica las variables más importantes, detecta patrones no triviales en la distribución de pago, manejo adecuado de missing values
Satisfactorio	60–74	EDA básico con estadísticas descriptivas y algunas visualizaciones, identifica el desbalance de clases, manejo mínimo de datos faltantes

Nivel	Puntos	Descripción
Insuficiente	< 60	EDA superficial o ausente, no detecta patrones clave, no maneja datos faltantes o lo hace incorrectamente

3.2 Modelo predictivo (40%)

Nivel	Puntos	Descripción
Sobresaliente	90–100	AUC > 0.70, supera el baseline significativamente, feature engineering creativo, validación temporal correcta, análisis de threshold óptimo, sin data leakage, múltiples algoritmos comparados con justificación
Notable	75–89	AUC entre 0.66–0.70, supera el baseline, al menos 2 algoritmos comparados, split temporal correcto, feature engineering básico, sin data leakage
Satisfactorio	60–74	AUC entre 0.639–0.66, supera marginalmente el baseline, al menos 1 modelo adicional, split temporal o cross-validation, sin data leakage evidente
Insuficiente	< 60	No supera el baseline, split aleatorio en vez de temporal, usa features post-hoc (data leakage), o no presenta métricas de evaluación

3.3 Propuesta de arquitectura (30%)

Los participantes deben proponer cómo llevarían su modelo a producción. No se requiere implementación; se evalúa la calidad del diseño y la justificación técnica.

Nivel	Puntos	Descripción
Sobresaliente	90–100	Propuesta completa: diagrama de arquitectura, endpoint de inferencia (API REST o batch), estrategia de reentrenamiento con detección de drift, pipeline de datos, monitoreo en producción, consideraciones de escalabilidad
Notable	75–89	Propuesta sólida con: método de serving (API o batch), frecuencia de reentrenamiento, pipeline de datos básico, mención de monitoreo
Satisfactorio	60–74	Propuesta básica que menciona cómo servir el modelo y cuándo reentrenar, sin mucho detalle técnico
Insuficiente	< 60	Sin propuesta de arquitectura, o propuesta sin fundamento técnico

3.4 Bonus: predicción de cartera incobrable (+10 puntos)

Reto adicional opcional: construir un segundo modelo que identifique, al momento de la solicitud, a los clientes que se espera que ya no paguen después de 240 días adicionales al plazo estipulado del crédito (cuenta incobrable).

Contexto de negocio: un crédito con más de 240 días sin abono después de la fecha de vencimiento se considera cartera incobrable. Identificar estos casos desde el otorgamiento permitiría tomar medidas preventivas (mayor enganche, aval obligatorio, rechazo del crédito).

Definición de la variable: `incobrable = 1 si exceso_dias > 240, de lo contrario 0`

El modelo debe usar las mismas restricciones que el modelo principal: solo features de originación, split temporal, sin data leakage.

Criterio	Puntos
Modelo funcional con AUC > 0.60 para clase incobrable	+4
Análisis del desbalance de clases (incobrables son minoría)	+2
Interpretación: ¿qué features predicen incobrabilidad?	+2
Propuesta de política de negocio basada en los resultados	+2

Máximo bonus: 10 puntos adicionales que se suman a la calificación final (máximo posible: 110/100).

3.5 Resumen de calificación

Componente	Peso	Puntos máx.
Comprensión de datos y EDA	30%	100
Modelo predictivo	40%	100
Propuesta de arquitectura	30%	100
Bonus: cartera incobrable	Extra	+10

Calificación máxima posible: **110 puntos** (100 base + 10 bonus).

---

4. Restricciones y reglas

1. **Split temporal obligatorio:** los datos deben dividirse cronológicamente por `fecha_entrega`, NO aleatoriamente.
2. **Sin data leakage:** solo usar las 20 features de originación entregadas y las que deriven de ellas mediante feature engineering. `exceso_dias` y `fecha_entrega` no son features.
3. **Métrica principal:** AUC-ROC (también se evalúa el recall de "No cumple" por relevancia de negocio).
4. **Reproducibilidad:** el notebook debe ser ejecutable de inicio a fin sin errores.
5. **Herramientas:** se puede usar cualquier librería de Python (scikit-learn, XGBoost, LightGBM, etc.).

6. **Entregable:** un notebook de Databricks con código, resultados y conclusiones.

---

5. Nota para todos los retos: tips y restricciones

Tip 1. Suban los datos a un volumen dentro de un esquema y un catálogo

En Databricks, los archivos del reto se gobiernan con Unity Catalog. Recomendamos esta estructura:

1. Crear (o usar) un **catálogo**, por ejemplo `datathon`.
2. Crear un **esquema** para el reto, por ejemplo `datathon.roga`.
3. Crear un **volumen** dentro del esquema, por ejemplo `datathon.roga.raw`.
4. Subir el contenido de `datos/` al volumen desde **Catalog > (catálogo) > (esquema) > (volumen) > Upload to this volume**.
5. Crear tablas Delta a partir de los CSV (con **Create table** en la UI, o con `read_files` en SQL) para usarlas en el EDA, el entrenamiento y MLflow.

Tener los datos en un volumen y en tablas de Unity Catalog facilita su trazabilidad, reutilización y gobierno en notebooks, modelos y experimentos.

Tip 2. Usen Genie Code

Genie Code es el asistente de IA integrado en Databricks. Les puede ayudar a:

- Explorar el dataset: nulos, distribuciones, desbalance de clases y asociaciones.
- Generar y depurar código SQL y Python para limpieza, feature engineering y entrenamiento.
- Implementar correctamente el split temporal y detectar posibles fuentes de data leakage.
- Registrar experimentos en MLflow y comparar algoritmos.
- Bosquejar y documentar la propuesta de arquitectura.

Sean específicos en sus instrucciones (tablas, rutas del volumen, resultado esperado) y validen siempre el resultado.

Restricción: límites de consumo de Databricks Free Edition

Databricks Free Edition tiene **límites de consumo diario** de cómputo. Esta restricción simula, de forma intencional, el uso de *budgets* y límites de consumo que existen en un entorno empresarial real: diseñen su solución pensando en eficiencia de costos.

Recomendaciones:

- Cada integrante del equipo puede usar su propia cuenta de Databricks Free Edition.
- Si una cuenta alcanza su límite diario, **muevan el código y los objetos** (notebooks, archivos, datos del volumen, tablas, experimentos) a la Free Edition de otro integrante para continuar.
- Mantener el código en un repositorio Git o exportar notebooks facilita este traslado.
- Escriban su solución de forma que se pueda recrear desde cero, lo que además suma puntos en reproducibilidad.
- Prefieran consultas acotadas y eviten dejar recursos encendidos sin uso.

¡Éxito a todos los equipos!